

→ Hoje → Heap Object Flow Graph (1)

↳ Focado em encontrar detalhes ems de memória que acontecem pelo uso impróprio dos privilégios de alocação dinâmica de memória em C e C++.

↳ Utiliza apenas os ponteiros nos quais o fluxo de memória dinâmica tem controle, ignorando demais variáveis e constantes que não são armazenamentos de memória, tais ponteiros são utilizados ~~durante~~ para geração das representações intermediárias → desse forma: eliminou o contexto ~~de~~ visualizado

↳ mapeia o fluxo de cada objeto alocado dinamicamente, do seu ponto de criação até a sua deslocação

- ↳ Holo → Continuação da introdução
- ↳ representação intermediária de um programa
- ↳ O programa é convertido para o formato SSA → realizado pois auxilia a diferenciar as atribuições e usos de cada variável/ponteiro
- ↳ Codo não também possui informações adicionais sobre o ponteiro → "embedding"
- ↳ A geração do Holo se divide em três

Canais:

- ↳ geração do Holo para o programa inteiro
- ↳ remoção dos pontos sem uso
- ↳ detecção e ~~código~~ otimização de caminhos com pontos detectados

↳ Nos: três tipos:

nos de objeto, nos de ponteiro e nos de divergência



→ H₀ → Continuação → criação do grupo (II)

→ as onças possuem informações sobre as condições ambientais para que aquele fluxo/percurso seja tomado

↳ cada onça possui o ponteiro para o vizinho para o começo e fim, bem como para uma lista dos ponteiros para estados condicionais que são restrições para a onça.

↳ as informações dos ponteiros já estão adicionadas no próprio grupo.

↳ Após a montagem do grupo, caminhos que não possuem vozamento são liberados.

↳ isto reduz o tamanho do grupo, eliminando os próximos passos de análise

↳ a remoção é feita obtendo um percurso de criação/colocação de um objeto de um estado de liberação, num percurso no qual as onças não possuem emoções de condicionais

↳ Hoq → continuação → graça e poder do go
↳ se o gofo Hoq resultante do poder não
possuir nenhum vértice ou aresta, ~~é~~ ~~verificado~~
é verificado que o programa não possui
argumentos de nomeio

(*) → adiciona característica neste trecho:

- ↳ ao remover itens durante a poob, apenas
raminhos em condicionais são removidos.
- ↳ com isso, arestas em condicionais ficam
sem vértice de origem.
- ↳ mudança: um vértice só deve ser
apagado se todos os seus vértices filhos,
em condicionais ou não, forem de fato
liberados. e, além disso, deve permitir entre
seus filhos um nó de liberação

(**) not: pensar sobre processo de remoção:

↳ é possível pagar itens ou o mais interessante
adiciona blocos de remoção, para sempre evitar remoção!

↳ Hoje a continuação a obtenção de look (II)

↳ a partir do grau pedido;

↳ Se o caminho não termina com um nó de liberação, é classificado como 'must-look'

↳ Se houver nó de liberação, porém apenas em caminhos com condicionais, então é classificado como 'may-look'

↳ Existe a possibilidade de não ~~verificar~~ ^{verificar} todas as condições de forma estática, por conta disso, tal classificação 'may-look' é utilizada[≠]

↳ → Uniao geral → cap 4.

↳ é implementado utilizando o LLVM framework, sendo compilado para a obtenção do linked de cada arquivo de forma individual e então são combinados utilizando o LLVM gold linker, que gera um único arquivo de código para o programa inteiro no formato .bc



↳ O algoritmo segue os passos:

↳ Zuerst die Holz der ~~ersten~~ Linsen

↳ para esse comando o link, que o sumário caso a mesma não esteja disponível

↳ alguns o sumário se estiver disponível.

Le svolgizze o sermoni de lungea

↳ O grupo Hofg é definido como: $\text{HOFG}(V, E)$
onde:

$\rightarrow E \rightarrow$ ~~ter~~ ~~os~~ ~~restos~~ de denominação F , ~~restos~~ de
de referência R , e ~~restos~~ de denominação
de percursor D

→ $V \rightarrow$ Objeto colocada no memoria D_1, D_2, D_3 ,
variáveis de programa $P, P_1^*, P_2^*, free, \dots$

↳ para a análise do programa, é necessária a geração do HOFG para o programa completo com o ~~o~~ componente chave dos diferentes módulos e arquivos.

↳ Holz → (p. 4, 2)

(VII)

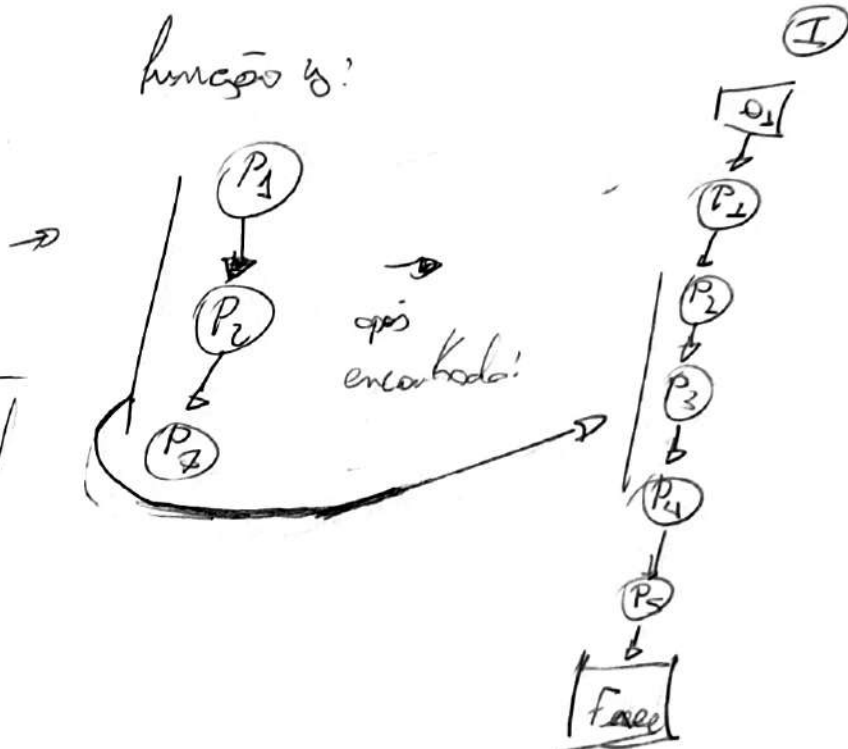
⊗ nota: modificação geração dos arquivos (.bc)
para que seja de longo período:

↳ arquivo A é removido por (.bc), o então
é obtida sua versão Holz.

↳ é possível que o mesmo precise de funções B, C etc
de outro arquivo/módulo.

↳ a partir disso, funções faltantes devem ser adicionadas
a lista de dependências, sendo removidas as obter
suas informações

Exemplo:



↳ Holz → 4.2 → continuação

(1211)

↳ Summaries de funções contém: a lista de argumentos, informações sobre itens locais e deslocador e valores retornados.

↳ a etapa de busca consiste numa DFS para cada nó do tipo objeto ^{como} ~~como~~ o começo do percurso ~~para~~ para encontrar caminhos do nó objeto ~~subto~~ do Holz do programa inteiro

↳ "Summary application" é a etapa de aplicação ~~do~~ ~~para~~ dos summaries das funções dentro do Holz global.

→ 4.2.2 → Analise sensível o contexto o caminho

↳ um nó do Holz é representado pelo tupla $\langle I_1, c, I_2 \rangle$, onde I_1 e I_2 são as instruções e o c é a condição do grafo de fluxo de controle que leva o item I_1 ao I_2

↳ condições que ~~podem~~ ~~se~~ resolvidas ~~por~~ ~~meio~~ do ~~CCUM IR~~ são gerados pelo CCUM IR

↳ Holg → 4.2.2 → Continuação

(IX)

↳ Cada campo de uma struct é mapeado pelo CCNA IR como uma variável distinta, isso permite que as instruções envolvendo tais campos sejam verificadas. Cada colocação e deslocação ~~para~~ ^{de} cada campo é distinto e mapeado do ~~como~~ ^{separado} singular.

↳ as arestas do grafo são direcionadas conforme o fluxo do programa. A ordem de execução de cada instrução é levada em consideração na fase de adição de custos do Holg, o que torna o análise sensível a fluxos.